

Overview:

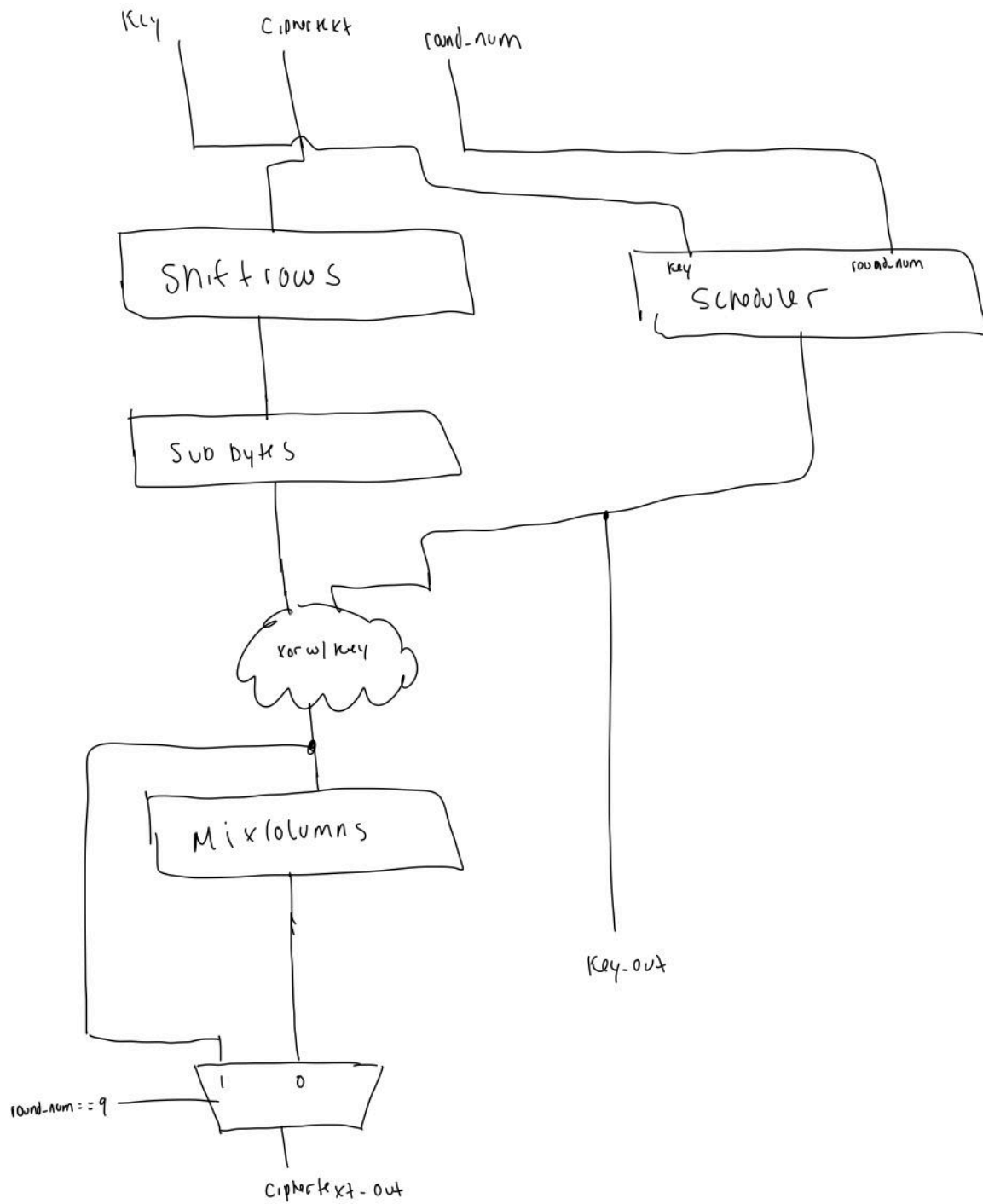
At this stage, I am choosing to make a “mini” AES implementation that will take in a 64 bit cipher text, and a 64 bit key and output the correct 64 bit plaintext. In order to minimize the area of my final product, I am planning on pipelining my implementation, meaning I will not create a module for each round, rather the decryption itself will take 10-cycles.

AES Modules:

My design will consist of the modules: ShiftRows, SubBytes & Sbox, MixColumns and Scheduler. The SubBytes module substitutes the bytes seen in the ciphertext depending on the appropriate output from the sbox. In my case, the sbox will essentially be a large lookup table. This can remain the same as a typical AES implementation. The ShiftRows module will perform a byte shift of each data row, depending on the index. The MixColumns module will multiply columns by a predetermined 4x4 matrix using operations in a Galois field. And the scheduler will perform operations on the current round's key, using the key and the round number to determine what the next key should be.

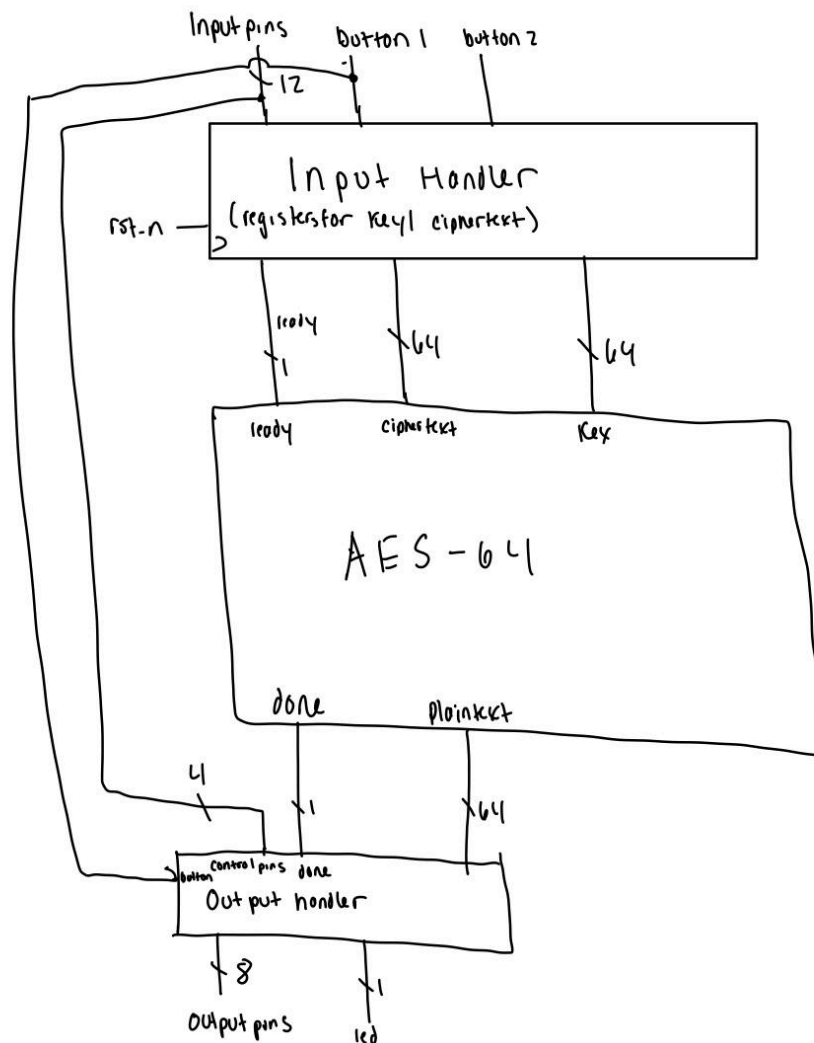
Overall Design layout:

This picture shows how each of these modules will be connected at a high level to complete one round of decryption. Note that as of right now, I am planning for this stage to be fully combinational. However this might change depending on area restrictions.

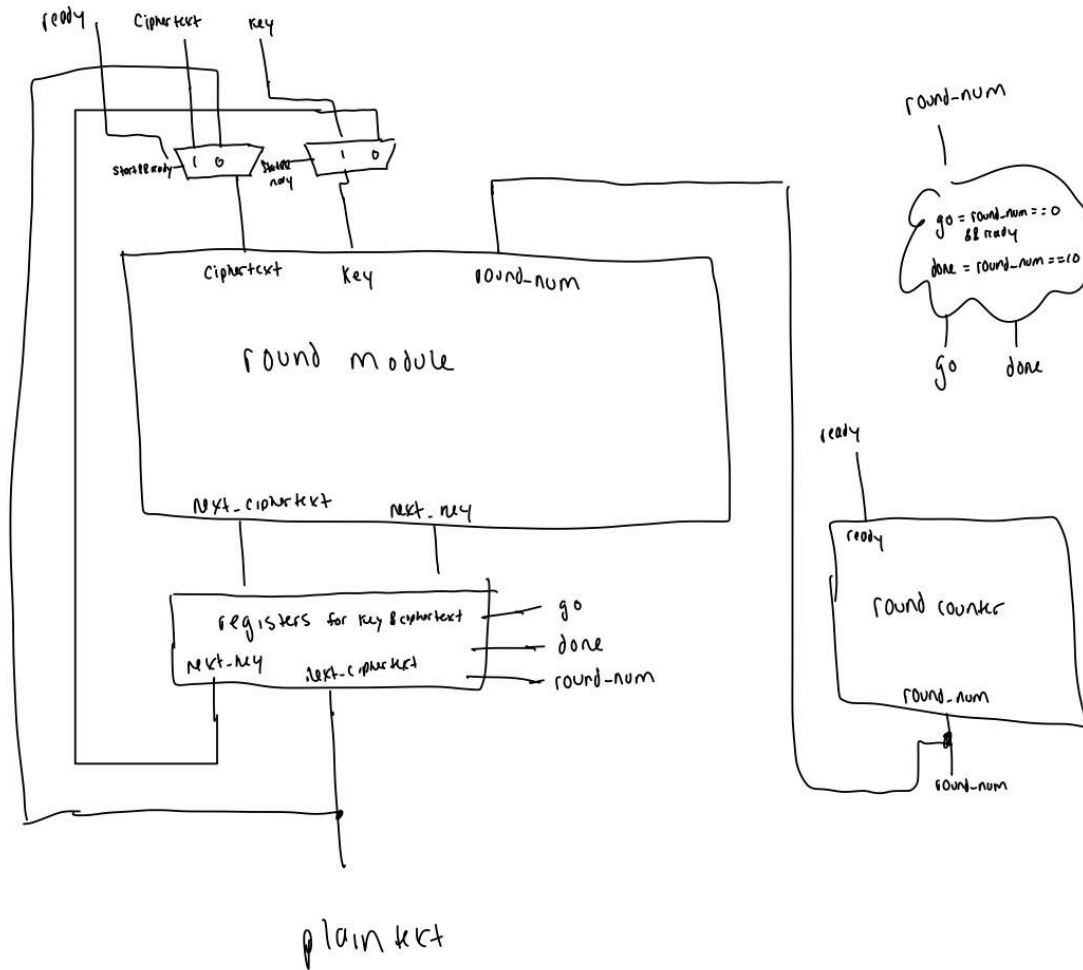


Top Level Design:

The current top level block diagram is shown below, once the input is ready, the decryption will begin from the first round, and continue until the counter reaches 10, indicating that decryption has finished. Once this happens, the ready signal will be asserted and this will indicate that the values at the output pins are correct. As of right now, to reduce logic, I am planning on using the chip's reset each time the user needs to compute a new value, but this might change in the future to be another on-chip button.



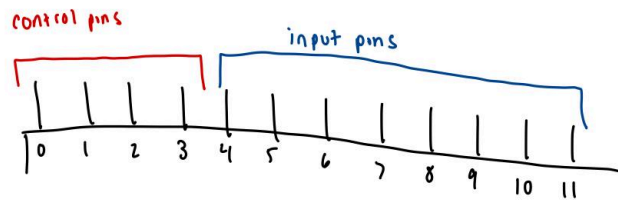
The image below shows the AES module in more detail. The counter functions like a control FSM. It starts at 0, and increments each cycle until it reaches 10. When the counter is equal to 10, the done signal is asserted. When ready is high and the counter is 0, the go signal will be asserted. This is used to feed the original inputs through the round module rather than the pipelined inputs.



Handling Inputs/Outputs:

The input/output modules shown in the top level design are left vague on purpose, as this is something that I have not completely worked out yet. Initially I was planning on taking in bare-metal inputs in a way that is similar to a scan chain: the user would put the values of the key and ciphertext (from MSB to LSB) on 8 of the input pins, then press the button to indicate that the value is correct, then I would shift them.

However, this seems like a method that would be prone to errors. Instead I was thinking of using another approach shown below, where the user can choose if they are inputting a byte for the key or the ciphertext, and can also overwrite previous inputs if there is a mistake.



inputs

- 0 → indicates if key or ciphertext
- 3:1 → indicate which byte of the key/ciphertext is being input
- 11:4 → input

Outputs

- 3:1 → unused for now
- 11:4 → output of module depending on selection from input pins (control pins)

I was considering using an established communication protocol such as SPI, however, I am not sure if I will have enough cells left to implement that as I am expecting my AES module to be fairly large. However, I plan on completing/testing my AES module first, before working on handling inputs/outputs, so I should have some time to change my plans if my AES module turns out to be smaller than I expect.

Testing:

To test this project I was planning on creating test vectors by hand. To my understanding, there should not be many edge cases for the AES-decryption module, so my plan was to make or find a golden module implementation in C or python, and check the outputs of my module against that. I also plan on testing each submodule individually, as I am working on them. This is much simpler however, so I think I will create these test vectors and their expected outputs by hand, and use simple assertions to check if they are as expected. Because I am planning on using a golden model, I will likely use CocoTB to check if the outputs match.

I will also need to test the modules that take in the inputs/outputs. This will be done using a basic SystemVerilog testbench, and I will use displays/simple assertions to make sure the input/output registers are correct, and that signals (such as the input ready/done signals are being asserted on the correct clock cycles).